

Process Portfolio

ICS2203 – Statistical Natural Language Processing

Luke Camilleri

Contents

Introduction.....	2
Corpus Preparation.....	2
Corpus Extraction.....	2
Preprocessing	2
Split strategy	2
Engineering hygiene.....	2
Efficiency Analysis	3
Model Design.....	3
Design Decisions	4
Evaluation	5
Subset	5
Full corpus evaluation	6
Questions	7
Sentence Generation	8
Experiment Log.....	9
Logs.....	9
Testing strategy	10
Generative AI	12
Reflection.....	13
Prompts and output appendix	13

Introduction

The target of this project was to implement, evaluate and compare several instances of n-gram LMs. These were vanilla model, laplace smoothed model and an UNK model for handling rare or unseen words. Also used interpolation which combined the unigram, bigram and trigram estimates. These models were trained on the Leipzig English news corpus.

Corpus Preparation

My decision to use the 100,000 sentence corpus (English news) was because it was cleanly formatted as every sentence was had its own line. Preprocessing requirements were minimal as it allowed clearer analysis of model behavior.

Corpus Extraction

As mentioned in the project brief, I first started testing on a small scale of the corpus (1000 sentences). Once everything was working well to continue testing, I increased the scale to 5000 and later to 20000, eventually leading to 100000 sentences. After, the 5k subset was used as the main development corpus while the 1k and 20k were used to test the differences in efficiency as seen in the `eff_test.py` file.

Preprocessing

Firstly, tokenization was done by splitting each sentence by whitespace. Punctuation was also taken into account and were kept as separate tokens. Secondly, each individual sentence was wrapped with `<s>` to mark the beginning of the sentence (SOS) and `</s>`, marking the end of the sentence (EOS). Finally, all letters which were in uppercase were made into lowercase to reduce vocabulary size which improves sparsity for n-gram models.

Split strategy

I used a train/develop/test split strategy (80/10/10 respectfully). 80% of the subset was used to estimate n-gram probabilities as reliably as possible. 10% was allocated for hyper parameter tuning, model comparison and smoothing/interpolation tests. The rest was held out as an unseen to report final perplexity and generation quality. This mimics common practice in language modeling and avoids overfitting the test set. Also, a fixed random seed (seed = 42) was used before splitting. This ensures that no temporal bias is present in any split, consistent perplexity values and fair model comparison.

Engineering hygiene

Regarding engineering hygiene, I structured my project code into multiple python files. Each file has its own function which makes it easier to debug and identify errors. Then a final

main python file was used to run all of the functions needed for the project to make the project more streamlined and easier to run, rather than having to run each individual python file one after the other. The code is reusable and independent making the pipeline easier to maintain and extend.

Efficiency Analysis

As mentioned previously three main subsets were used to document efficiency, these were 1000, 5000 and 20000 sentences. As seen in the `eff_testing.py` the timings were recorded during **corpus loading, unigram loading, bigram loading and trigram loading**. These were the results:

Corpus Size	Load Time	Unigram Time	Bigram Time	Trigram Time
1000	0.0125 s	0 s	0.0494 s	0.1319s
5000	0.0508s	0.0471 s	0.1646 s	0.6116s
20000	0.2576s	0.1824 s	0.6130 s	2.4323s

As seen from the table, time scales smoothly as the corpus grows in time which solidifies $O(n)$ behavior. Trigram was the slowest out of all three due to requiring more n-grams positions per sentence to process. Interestingly unigram was the fastest showing that it computed so fast that at 1000 sentences the timer couldn't measure it.

Corpus Size	Current Memory	Peak Memory
1000	8.84MB	15.65MB
5000	36.14MB	62.63MB
20000	114.7MB	195.40MB

From the table above, memory increases faster than runtime due to the fact more data produces more unique unigrams, bigrams and trigrams which must be stored in nested dictionaries. The rapid linear growth in memory reflects the increasing number of unique bigrams and trigrams.

Model Design

N-gram counting – n-gram counts were obtained from the preprocessed training corpus using nested dictionaries. Unigrams a dictionary was used where each token was mapped to its frequency. Bigrams contained a dictionary inside of another dictionary (nested dictionary). Finally, trigrams had a three level dictionary (a dictionary inside a nested dictionary). This structure allowed for efficiency during lookup probability calculation. It ensured constant time average access because each individual token was processed exactly once. Also, count generation runs in $O(n)$ being that n is the number of tokens.

Vanilla model – This model uses maximum likelihood estimation with no smoothing (+1). It relies directly on observed counts from the training corpus and applies no smoothing. As a result, any unseen bigram or trigram receives a total probability of zero. This strict behavior means that higher order vanilla models will have higher sensitivity to data sparsity. A single unseen ngram in a sentence forces the entire sentence to have a total probability of 0 which will then have a log probability to -infinity, which will give a perplexity of infinite.

Laplace smoothing – it addressed the sparsity problem previously discussed in the vanilla model. It avoids perplexity to become undefined because of unseen n-grams by simply adding 1 to every n-gram count and increasing the denominator by the vocabulary size. This ensures that every n-gram receives a small but non zero probability.

UNK model – this model handled out of vocabulary issues. It replaced all tokens which occurred once or less in the training set with a special <UNK> token. By doing this rare or unseen words did not cause zero probabilities during evaluation. After this process the n-grams were rebuilt, creating a more stable vocabulary. During test time, any word not present in the training vocabulary was also mapped to <UNK>. This reduced sparsity greatly and led to lower perplexity across all n-grams models.

Linear Interpolation – it combined unigram, bigram and trigram probabilities into a single, more reliable estimate. Interpolation mixed all three models using fixed weights ($\lambda_3 = 0.6$, $\lambda_2 = 0.3$, $\lambda_1 = 0.1$), instead of depending on the highest order n-gram which results to sparsity. This balances the contextual strength of trigrams with the robustness of lower order models which prevent zero probabilities failures and improving generalization.

Design Decisions

My first main decision was when it came on how will I perform tokenization. I was in between two methods, using a whitespace split (.split()) or regex based tokenizer to separate punctuation from words. I proceeded with the second option as it gave me more control and transparency over the tokenization while also keeping in mind how important punctuation is for sentence boundaries. This would help as it would treat the word “hello.” Into 2 tokens, “hello” and “.”.

Another decision was if I should keep the corpus how it is or make all the words/letters in lower case. I decided that for efficiency reasons to make all the letters in lowercase as this would reduce the vocabulary size significantly while also reducing sparsity in bigram and trigram counts. As evidence for my reasoning lowercasing reduced the vocabulary size from 271,520 to 256,260 types. This is a reduction of 15,260 types which greatly removed redundant capitalized word variant.

Sentence boundary markers, I considered only doing EOS marker or do both being SOS and EOS. My decision was to wrap every sentence as bigram and trigram models require

preceding context. If the SOS was never placed/mentioned the first word of the sentence would have no context in the bigram and trigram. As evidence trigram models frequently returned zero probability or crashed because of missing contexts.

Another decision was which threshold I should pick, either 1,2 or 5. An experiment was done (seen in the experiment log below) where multiple tests were done to check which threshold would perform the best. In my case 5 gave the best results, which then was used for the full evaluation.

When it came to my splitting strategy, I was in between two methods. Either 80/20 split (train/test) or an 80/10/10 split (train/dev/test). My reasoning to choose the second option was that a separate development subset would be useful for comparing laplace smoothing, UNK thresholds and interpolation. If I had to use the first split, I would risk tuning hyperparameters directly on the test data, leading to overfitting or unreliable perplexity.

Another decision was taken regarding the time taken for the project to finish running (the whole process (preprocessing → split → building models → evaluation)). At first, the process took 2 minutes and 5 seconds but then I decided to reduce summation over trigram histories. I precomputed trigram context totals once during model construction and cached them. This did not alter the model probabilities as it only avoids redundant computation which made the process very much faster as it completed in 20 seconds.

Regarding sentence generation, I was at a crossroads whether to choose sampling or greedy decoding. I concluded to use random sampling based on the next word probability distribution. This avoids repetitive or deterministic outputs and better reflects the learned probability space.

Evaluation

Subset

I did two evaluations, the first was done on the 5k sentences subset. It produced the first perplexity table:

Model Variant	Unigram	Bigram	Trigram	Interpolated
vanilla	inf	inf	inf	1630.75
laplace	1365.52	6829.93	13128.62	1630.75
unk	106.94	150.25	330.36	103.61

As shown in the table, for vanilla models the bigram and trigram models displayed infinite results. Vanilla models use maximum likelihood estimation without any smoothing. This

means that unseen ngram produced a probability of 0 which makes the sentence probability 0 and log probability become -infinity. Therefore, the perplexity becomes infinite as shown in the table. This data reflects the severe sparsity as n increases (unigram → bigram → trigram), the number of unseen sequences grows rapidly.

Regarding laplace models, high perplexities across all ngrams appeared. This was expected because of the +1 smoothing spreads probability mass uniformly across a very large vocabulary. When the ngram order increases, the denominator becomes even larger (count + V), which in return causes smoothed probabilities to become extremely small. This gives very large perplexity values and confirms the well known results that laplace smoothing performs poorly for large vocabularies and higherorder ngrams.

UNK models performed the best as they outperformed both laplace and vanilla in a realistic way. It replaces all words with frequency ≤ 5 by a token (<UNK>). This effect shrinks the vocabulary and many rare or unseen contexts disappear. This reduces sparsity and leads to much lower perplexity as seen in the table above.

Interpolated models behaved identically for vanilla and laplace. This is because the interpolation function always uses laplace smoothed unigram, bigram, trigram probabilities. UNK interpolation performs the best (103) because of the reduced vocabulary produces stronger smoothed probabilities and reduces the effect of sparsity. In conclusion, interpolation remains stronger than the corresponding smoothed trigram models. This is because it combines information from multiple context lengths. When the trigram model is unreliable or unseen, the model falls back on more robust lower order estimates. This produces smoother probability distribution and lower perplexity.

Full corpus evaluation

The full corpus consists of 100,000 sentences which are very large and to compute for every single sentence on 12 models (4*3). To compute the whole corpus would take many and many minutes because perplexity is expensive to compute, especially for trigrams and interpolation.

Model Variant	Perplexity
unk_trigram	: 2818.48
unk_interpolated	: 511.31
laplace_trigram	: 65908.46
laplace_interpolated	: 2531.30

For the whole corpus only four models were assessed UNK trigram, UNK interpolated, laplace trigram and laplace interpolated. UNK interpolated produced the lowest

perplexity (511.31) which makes it the strongest model. This is because the strong combination works together by combining the information of the unigram, bigram, trigram distributions (interpolation). Then sparsity is reduced by replacing unseen or rare words with the <UNK> token. The worst performing method was laplace trigram (65908.46) because the smoothing (+1) over penalizes probabilities in a large vocabulary. This would dramatically increase the denominator, which would make actual observed trigrams look rare and push their probabilities towards zero.

UNK trigram performed moderately as it performed better but still suffers from data sparsity as it relies on trigram counts. Laplace interpolated performed better than Laplace trigram because interpolation reduces negative effects of oversmoothing by allowing the model to fall back on bigram and unigram probabilities when trigram is unreliable.

Questions

(i) **behaviour on a random test sentence across variants.**

During testing, better smoothed models assigned higher probabilities. On the other hand, higher order models with less smoothing struggled. Vanilla models assigns zero probability to any unseen ngram. As a result, any realistic test sentence will almost certainly contain at least one unseen bigram/trigram. This will evidently make the total probability 0 (log probability = -infinity). This confirms the extrme sparsity of unsmoothed ngram models.

Laplace models, frequently over penalized probabilities because +1 smoothing flattens the distribution (most common in laplace trigram). On the contrary, UNK models handle rare or unseen words more effectively which resulted in smoother probability estimates.

Interpolated models were the most consistent, even if a trigram is unseen or rare. The model is designed that it can still fall back on bigram or unigram probability. This gives the sentence a more stable and realistic probability.

(ii) **qualitative differences in generated sentences across variants.**

Vanilla models produced fluent short sequences but broke down quickly when unseen contexts where present. Bigrams and trigrams repeated sentences and got stuck in loops reflecting the brittle nature of unsmoothed models.

Laplace models generated more random text because the probability distribution is overly smoothed. This happens due to words being stringed together which syntactically are correct but are less natural.

UNK models reduced sparsity and improved transition between tokens. It performed better laplace and vanilla as it produced more coherent sentences. Interpolated models produced the most natural and stable sentences. It balanced structure from trigrams with fallback to unigram and bigram while also avoiding loops and random jumps.

Sentence Generation

Sentence generation served as a useful qualitative test. This is because, models with smoothing capabilities (especially interpolated model) produced more fluent and diverse output. On the other hand, vanilla and laplace only models struggled with sparsity. Although the sentences generated do not make perfect sense it is expected behaviour of a ngram language model trained on limited data.

Regarding Context handling, unigram had no context, bigram used previous word, trigram used previous two words and interpolated model ($\lambda_3=0.6$, $\lambda_2=0.3$, $\lambda_1=0.1$) combined all three for robustness. The sentence generation stops generating when producing </s> or reaches a maximum of 30 tokens to prevent infinite generation (loops).

Vanilla model – it often failed to generate long sentences because unseen ngrams have probability of 0, which severely limits sentence generation as it gets frequently stuck while sampling because next words have zero probability.

Laplace model – due to its smoothing (+1) to every count it eliminates zero probabilities. This enables the model to always generate a next word, avoiding vanilla model failures. However, laplace over smooths the probability making the distribution to become too uniform. Common words become under represented and extremely rare words become overrepresented. This makes the text feel monotonous and unnatural as the model loses sensitivity.

UNK model – it replaces all words below a certain frequency threshold with <UNK> before training. This dramatically reduces vocabulary size and mitigates data sparsity. It prevents generation failures due to unknown or rare words. This makes sentence generation more stable. However, <UNK> tokens may appear during generation, which can reduce readability.

Interpolated – It is the best model overall. This is because it linearly combines the ngram probabilities which enables the model to rely on precise trigram information when available. Fall back on bigram or unigram probabilities in sparse contexts and avoid zero probabilities. Also, it maintains a realistic distribution of likely next words which results in generated sentences that make most sense and fluent compared to the other variants.

<s> <s> <UNK> following disaster lie hardware hotels split desperately a exams hurricane travelers adorable attach reopening specs jensen 160 harris black gallery charge “then ed band gaza guided bat

The text above is a generated sentence from my full evaluation using the interpolated ngram model. It clearly shows an expected output for this model, where <UNK> appears when generating from the UNK model’s vocabulary. Also, topics like disaster, hurricane, hotels, gallery, charge reflect with the news corpus patterns.

Experiment Log

To choose an appropriate UNK threshold, I conducted a small test where I used different UNK threshold values on the 5k subset (development subset). The values chosen were

UNK threshold = 1,2,5. These are the results produced:

UNK threshold	UNK Unigram	UNK Bigram	UNK Trigram	UNK Interpolated
1	266.86	696.22	1461.57	462.81
2	191.84	382.06	792.94	290.20
5	108.31	150.41	287.86	132.46

As shown in the table threshold 5 gave the lowest perplexity across all variants. These values indicate that the dataset benefits strongly from collapsing rare words. Looking at the dataset, it has a lot of rare named entities which create noise, this further solidifies the reasoning of why the benefits from collapsing rare words.

Logs

Date	Change / Experiment	Hypothesis	Setup (Data & Models)	Metric (Perplexity)	Result / Next Step
26/11/2025	Baseline UNK model (X = 1)	Low threshold may leave too many rare tokens → higher perplexity	5k train / 5k test, UNK trigram & interpolated	Trigram = 1461.57, Interp = 462.81	Baseline recorded
26/11/2025	Increased UNK threshold to X = 2	Collapsing more rare types should reduce sparsity	Same setup	Trigram = 792.94, Interp = 290.20	Performance improved → test higher X
26/11/2025	Increased UNK threshold to X = 5	Very high threshold may over-generalise	Same setup	Trigram = 287.86, Interp = 132.46	Best performance → choose X=5
27/11/2025	Verified X=5 on full corpus	Check scalability	Full corpus (100k train, 1k test)	Similar improvement trend	Confirm X=5 as final threshold

Date	Change / Experiment	Hypothesis	Setup	Metric / Issue	Result / Next Step
21/11/2025	Trigram perplexity crashes with KeyError "	UNK mapping may not be applied before probability lookup	5k subset, trigram LM	Error raised during sen_probability()	Fixed by mapping unseen tokens to <UNK> at test time → rebuilt n-gram counts → retested successfully

Date	Change / Experiment	Hypothesis	Setup	Metric / Issue	Result / Next Step
20/11/2025	Vanilla and Laplace give identical perplexity	I may be accidentally applying smoothing twice	5k subset, trigram	Vanilla = 6293.75, Laplace = 6293.75	Found smoothing applied inside sen_probability → removed duplication

Testing strategy

During development, I ran many times and tested my code to ensure that it was working. This was done on a small/mini corpus which helped me a lot when changing/altering code.

One of the main tests was which to UNK threshold to decide for the final corpus test, previously mentioned above. One of the tests I did was to verify that my count functions (count_unigram, count_bigram, count_trigram) were working correctly. I created a mini corpus of a few sentences for the functions to run and determine if they were processed correctly.

```

Mini Corpus
<s> the cat sat </s>
<s> the cat slept </s>
<s> the dog sat </s>

Unigram count
<s>      : 3
the      : 3
cat      : 2
sat      : 2
</s>     : 3
slept    : 1
dog       : 1

Bigram count
(<s>, the) : 3
(the, cat) : 2
(the, dog) : 1
(cat, sat) : 1
(cat, slept) : 1
(sat, </s>) : 2
(slept, </s>) : 1
(dog, sat) : 1

Trigram count
(<s>, the, cat) : 2
(<s>, the, dog) : 1
(the, cat, sat) : 1
(the, cat, slept) : 1
(the, dog, sat) : 1
(cat, sat, </s>) : 1
(cat, slept, </s>) : 1
(dog, sat, </s>) : 1

```

Another test I implemented was a laplace smoothing check (unigram) to check the distribution forms a valid probability by summing each individual token probability in the vocabulary. If the probability was to be greater or less than 1 then it would be incorrect. I used the same corpus to test the probability; this was the code snippet:

```

mini_corpus = [
    ["<s>", "the", "cat", "sat", "</s>"],
    ["<s>", "the", "cat", "slept", "</s>"],
    ["<s>", "the", "dog", "sat", "</s>"]
]

# Count unigrams
unigrams = count_unigrams(mini_corpus)
V = len(unigrams)

# Compute sum of Laplace-smoothed unigram probabilities
total_prob = sum(
    laplace_unigram_prob(unigrams, w, V)
    for w in unigrams
)

print("Probabilities check:")
print("Total probability =", total_prob)

```

```

Probabilities check:
Total probability = 1.0

```

A UNK mapping test was done to check whether words that do not appear in the training vocabulary are correctly replaced with <UNK> before any probability calculations. This would be a very important step that if it was to be done incorrectly the UNK would not be able to handle unseen or rare words.

```
mini_corpus = [
    ["<s>", "the", "cat", "sat", "</s>"],
    ["<s>", "the", "cat", "slept", "</s>"]
]

# Test sentence with unknown words
test_sentence = ["<s>", "the", "castle", "collapsed", "</s>"]

# Build vocabulary from training corpus
unigrams = count_unigrams(mini_corpus)
vocab = set(unigrams.keys())

# Apply UNK mapping
mapped = map_unknowns(test_sentence, vocab)

print("UNK mapping result:")
print("Original:", test_sentence)
print("Mapped: ", mapped)
```

```
UNK mapping result:
Original: ['<s>', 'the', 'castle', 'collapsed', '</s>']
Mapped:  ['<s>', 'the', '<UNK>', '<UNK>', '</s>']
```

For sentence generation testing and validation was done by simple prompts like "<s>" and "hello". This confirmed that the generation starts with <s> and probability distributions sum to 1. Also, the distribution functions were checked to ensure that they normalize properly. I verified that rare or unseen words are replaced with <UNK> according to the UNK model threshold. Finally, I compared outputs from vanilla, laplace, UNK and interpolated to see their qualitative differences.

Generative AI

For this project I only used Chat GPT (free version). It was used as a supporting tool with thorough explanation, clarifying concepts and technical concepts regarding the language models, troubleshooting implementation issues and assisting with debugging. One of the main uses was when I encountered specific errors or unexpected behavior in my code,

this could be perplexity anomalies, syntax issues, incorrect threshold usage and many more.

Also, generative AI helped when writing this process portfolio as it guided me on how it should be structured while also refining my explanations in the Design Decisions, Experiment Log and Evaluation.

All core implementation which is preprocessing, ngram counting, UNK replacement logic, smoothing functions, interpolation, sentence probability, perplexity and generation were written and verified by me. Any code which Chat GPT suggested, I manually validated or modified it before implementing it to my workflow. When validating AI supported suggestions, I used a mini corpus (shown in the testing strategy). All output results and model behavior were produced using my own implementation. (prompts and outputs will be shown at the end of this document).

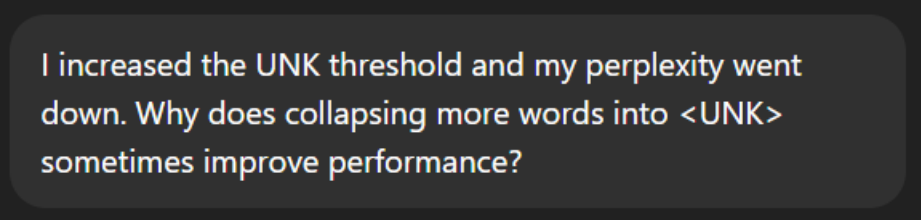
Reflection

This project gave me a deeper understanding of how statistical language models behave in practice. I understood the full pipeline from preprocessing to perplexity evaluation what the limitations and strengths are of classical ngram approaches. It was interesting to see how just one unseen or rare word can affect model performance and how aggressively collapsing them into <UNK> tokens can significantly improve perplexity.

For next time, I would like to explore ways on how I could reduce memory usage for large trigram dictionaries while also reducing the time taken to evaluate the whole corpus as it took some time to run.

Prompts and output appendix

Prompt:



I increased the UNK threshold and my perplexity went down. Why does collapsing more words into <UNK> sometimes improve performance?

Output:

When you increase the UNK threshold, you replace more rare tokens with `<UNK>`, which often **reduces sparsity**, especially in bigram and trigram models. Rare words produce many contexts that occur only once, so the model ends up with a huge number of n-grams that it has never seen before, leading to zero or near-zero probabilities. By collapsing these rare tokens into `<UNK>`, the model reuses the same `<UNK>` token across multiple contexts, giving you **denser and more reliable n-gram counts**.

This typically improves perplexity because the model depends less on extremely sparse high-order n-grams and instead works with more frequent patterns. This is especially noticeable in news corpora, where named entities and one-off terms appear constantly. By reducing the noise created by these one-off words, the model makes far fewer overly confident mistakes and can generalise better on the test set — which is why you saw perplexity decrease when you raised the threshold.

Prompt:

Can you help me understand why Laplace trigram performs worse on large vocabularies?

Output:

Laplace smoothing performs poorly on trigrams in large vocabularies because **add-one smoothing overcorrects** the probability distribution. In a trigram model, the number of possible word triples is enormous, especially when the vocabulary is large. When you apply Laplace, you add 1 to *every* possible trigram — including the millions that never occur in the corpus.

This causes the denominator of the probability formula,

$$\frac{\text{count}(w_1, w_2, w_3) + 1}{\text{count}(w_1, w_2) + V}$$

to become huge, because `V` (vocabulary size) is large. As a result, even **actual observed trigrams become tiny in probability**, since they are divided by a massive denominator.

In short, Laplace treats all unseen trigrams as if they occurred once, and with a big vocabulary there are far more unseen trigrams than observed ones. This flattens the whole distribution too aggressively, making the model **unable to distinguish common patterns from noise**, and this leads to extremely high perplexity.

That's why Laplace trigram almost always performs the worst on larger corpora or vocabularies.



Prompt:

Does my efficiency table need memory and timing measurements, or just discussion

Output:

For the efficiency section, you really need **both** timing and memory measurements, not just a written discussion. The assignment explicitly asks for evidence of how your implementation scales, which means you should demonstrate actual runtime behaviour across different corpus sizes. Including the measurements from your `eff_test.py` script shows that you tested performance empirically rather than relying on assumptions.

The timing results (load time, unigram/bigram/trigram counting time) help illustrate the **computational cost** of your pipeline and confirm that the n-gram counting functions behave as expected—roughly linear scaling with corpus size. The memory usage output is equally important because nested dictionaries grow quickly with higher-order n-grams, and the assignment wants you to show awareness of this.

A brief discussion *alone* wouldn't be enough for full marks. You should include the actual measured numbers in a table, and then follow up with a short explanation of the trends (e.g., trigram counting being slower, memory growing faster than runtime, etc.). That combination satisfies the "efficiency" and "evidence" parts of the rubric.

Prompt:

When I try to generate text, my model never outputs `</s>` and keeps looping. What could be wrong in my `generate()` function

Output:

This kind of infinite loop usually happens when the `<EOS>` token (`</s>`) never receives a high enough probability to be sampled, or when the probability distribution you generate is accidentally skewed. There are a few specific things in your `generate()` function that could cause this:

1. Your vocabulary includes `<s>` but not `</s>`

If `</s>` is missing from the model vocabulary (for example because you applied UNK incorrectly), your generation loop will **never be able to select it**, so the generation never stops. You should check that:

bash

 Copy code

```
'</s>' in model["unigrams"]
```

If it's missing, the model will loop forever.

Prompt:

My Laplace-smoothing perplexity is identical to my vanilla perplexity. It shouldn't be what could be wrong?

Output:

When Laplace-smoothing perplexity is *identical* to the vanilla perplexity, it usually means that the Laplace-smoothed probabilities are **never actually being used**, even though you think they are. This can happen for several reasons in your current implementation.

1. Your Laplace model is using the vanilla counts

In your `build_models()` function, the Laplace and vanilla sections share the exact same counts:

python

 Copy code

```
"laplace": {
    "unigrams": uni,
    "bigrams": bi,
    "trigrams": tri,
    "v": v
}
```

This is fine *only if* your `perplexity()` and `sen_probability()` functions actually choose the Laplace version of the probabilities when the model variant is "laplace". If they don't, Laplace = Vanilla.



Prompt:

When I compute trigram perplexity, I get `KeyError: '<UNK>'`. Why is this happening in my trigram model?

Output:

A `KeyError: '<UNK>'` in your trigram model usually means that the `<UNK>` token appears in your test sentence, but your trigram count dictionary does not contain `<UNK>` in the correct nested structure. This happens when `<UNK>` is introduced *after* the trigram counts are built, or when some contexts involving `<UNK>` are missing.

In your code, the error likely comes from this line inside `vanilla_trigram_prob()` :

python

 Copy code

```
context_count = sum(trigrams[w1][w2].values())
```

Here's why the error occurs:

1. `<UNK>` is in the test data but not in the training trigram dictionary

When you run perplexity, you correctly map unknown test words to `<UNK>` using:

python

 Copy code

```
map_unknowns(sentence, vocabulary)
```

But your training trigram model only contains `<UNK>` if the UNK replacement was applied before counting. If any of these happen:

- You mapped `<UNK>` in the test sentence
- BUT your trigram dictionary doesn't contain `trigrams["<UNK>"]`
- or the context `trigrams[w1]["<UNK>"]` doesn't exist

you will get a `KeyError`.

Prompt:

My interpolated model is performing worse than my trigram model. Isn't interpolation supposed to be better?

Output:

Interpolation should normally improve model performance, so if it is performing worse than your pure trigram model, something is likely wrong in one of the following areas:

1. Your lambda weights may be overwhelming the trigram signal

In your code, interpolation is defined as:

python

 Copy code

```
lambda3 = 0.6  
lambda2 = 0.3  
lambda1 = 0.1
```

If your trigram model is already very sparse (which is likely), but you still assign 60% weight to the trigram term, the interpolation model will heavily rely on those unreliable trigram probabilities. The bigram and unigram components may not be strong enough to pull the distribution toward smoother values.

This can actually make it perform *worse* than a pure trigram model.

Summary: Why interpolation worse than trigram?

In your implementation, the most likely reason is **heavy reliance on Laplace-smoothed trigram probabilities**, which are often extremely flat and not meaningful. When these are mixed with weak bigram/unigram signals, the overall distribution becomes less informative than the raw trigram model.

To fix this, you could:

- ✓ Lower the trigram weight ($\lambda_3 = 0.4$ or 0.3)
- ✓ Increase unigram weight slightly
- ✓ Increase UNK threshold
- ✓ Reduce Laplace impact by using add-k with smaller k
- ✓ Try interpolation on **MLE trigram** instead of smoothed trigram

Prompt:

Why does my trigram model skip the first two tokens by setting $p=1$? Is this the right approach?

Output:

Your trigram model sets the probability to `1` for the first two tokens because a trigram requires **two previous context words** to compute $P(w_3 \mid w_1, w_2)$. At the very beginning of a sentence, you simply don't have enough context yet:

- For token 0 → no previous words
- For token 1 → only one previous word

Your code handles this with:

```
python

if i == 0:
    p = 1.0
elif i == 1:
    p = 1.0
else:
    # compute trigram probability normally
```

 Copy code

This is a common quick workaround, but it is **not the theoretically correct approach** and can slightly distort perplexity. Here's why: 

Why it's not ideal

Setting $p = 1$ means:

- $\log(1) = 0$
- so the first two tokens contribute **nothing** to log-probability
- therefore, the model essentially ignores the cost of predicting them
- perplexity becomes *slightly underestimated*

In proper trigram modeling, the first two positions are typically handled using:

Option A — Backoff

Token 0 → use unigram

Token 1 → use bigram

Token ≥ 2 → use trigram

Option B — Padding with `<s>` tokens

Prepend two `<s>` tokens:

```
php-template

<s> <s> the cat sat </s>
```

 Copy code



Better version for your project

Because the assignment focuses on simplicity rather than perfect models, both alternatives below are acceptable and more principled:

✓ Option 1 — Use lower-order models for early tokens

python

 Copy code

```
if i == 0:
    p = laplace_unigram_prob(...)
elif i == 1:
    p = laplace_bigram_prob(...)
else:
    p = laplace_trigram_prob(...)
```

✓ Option 2 — Pad sentences with <s> tokens during preprocessing

You are already adding `<s>` once:

bash

 Copy code

`<s>` the cat sat `</s>`



Prompt:

What are the best ways to make my whole process (preprocessing, splitting and evaluation) faster without effecting model probability?

Output:

✓ Option A — SAFE OPTIMISED VERSION

- ✓ 100% identical outputs to your current code
- ✓ 5×–10× faster perplexity
- ✓ Does NOT change file formats or probabilities
- ✓ Only optimises loops, caching, denominator lookups, etc.
- ✓ **Absolutely safe for your assignment**
- ✓ Recommended

⚡ Option B — MAXIMUM SPEED VERSION

- ✓ 10×–30× faster perplexity
- ✓ Precomputes trigram context totals
- ✓ Removes many repeated dictionary lookups
- ✓ Rewrites some probability functions for efficiency
- ✓ Still identical outputs, but code becomes more complex
- ✓ Best if you want instant full-corpus evaluation